Práctica N°2: Circuitos Combinacionales y Secuenciales

Índice

1. Creación de archivos `.vhd` en Vivado para un proyecto por bloques funcionales **2**
- 1.1. Estructura de archivos recomendada . . . 2
- 1.2. Añadir o crear fuentes VHDL . . . 3
- 1.3. Designar el archivo principal (*Set as Top*) . . . 5

2. Descripción del Circuito en VHDL **5**
- 2.1. Módulo 1: Sumador con signo . . . 6
- 2.2. Módulo 2: Decodificador Hexadecimal a Display de 7 Segmentos 9
- 2.3. Módulo 3: Driver Multiplexor para Displays de 7 Segmentos . 12

3. Archivo principal: Integración de módulos y uso de `port map` **17**

4. Diagrama de bloques y modularidad del diseño **21**
- 4.1. Bloques funcionales individuales . . . 22
- 4.2. Ventajas de la modularidad . . . 23

5. Asignación de pines en la placa Nexys A7 100T **23**
- 5.1. Señales de entrada . . . 24
- 5.2. Señales de salida . . . 24
- 5.3. Consideraciones adicionales . . . 25

6. Resultados en la placa Nexys A7-100T **25**

Objetivos

Objetivo General

Implementar un circuito sumador con signo en lenguaje VHDL, utilizando el entorno de desarrollo Vivado, que muestre en un display de 7 segmentos los operandos y el resultado, incluyendo la indicación del signo de cada valor.

Objetivos Específicos

1. Comprender los conceptos fundamentales de diseño digital aplicados a circuitos combinacionales y secuenciales.
2. Analizar la estructura modular de un proyecto en VHDL, separando la funcionalidad en múltiples archivos para mejorar la organización y reutilización de código.
3. Aplicar el uso del *port map* para interconectar módulos diseñados de forma independiente.
4. Desarrollar el código VHDL para un sumador con signo, garantizando el manejo correcto de operandos positivos y negativos.
5. Implementar la decodificación y visualización de números con signo en un display de 7 segmentos.
6. Validar el funcionamiento del sistema en la tarjeta Nexys mediante pruebas con diferentes combinaciones de operandos.

1. Creación de archivos `.vhd` en Vivado para un proyecto por bloques funcionales

Esta sección te guía, para crear y organizar correctamente las fuentes VHDL en Vivado cuando trabajas con varios bloques funcionales (módulos auxiliares) y un módulo principal (*top*).

1.1. Estructura de archivos recomendada

Usa nombres consistentes y explícitos. Propón una estructura mínima como:

- **Principal (Top)**: `signed_adder.vhd` o `top_proyecto.vhd`
- **Auxiliares**: `sum_sign.vhd`, `hex2sseg.vhd`, `disp_mux.vhd`

1.2. Añadir o crear fuentes VHDL

Con el proyecto creado, se crean los archivo **.vhd** en los cuales se programará el codigo con la logica requerida, para ello se debe utilizar el simbolo '+' ubicado en el menú de recursos y seguir el procedimiento explicado en la practica anterior.

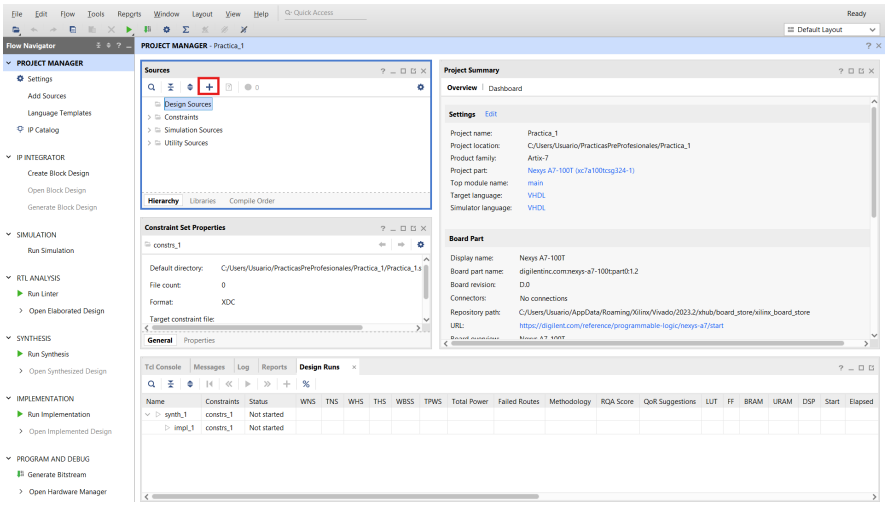


Figura 1: Creación y adición de archivos VHDL al proyecto.

Como se trabajarán con bloques se deben crear y de manera opcional definir las entradas y salidas de cada código, esto no es un paso obligatorio pero se sugiere tener claro los parámetros de cada bloque. Estos se pueden crear de uno a la vez como se observa en la Fig 2 o de manera conjunta como en la Fig 3.

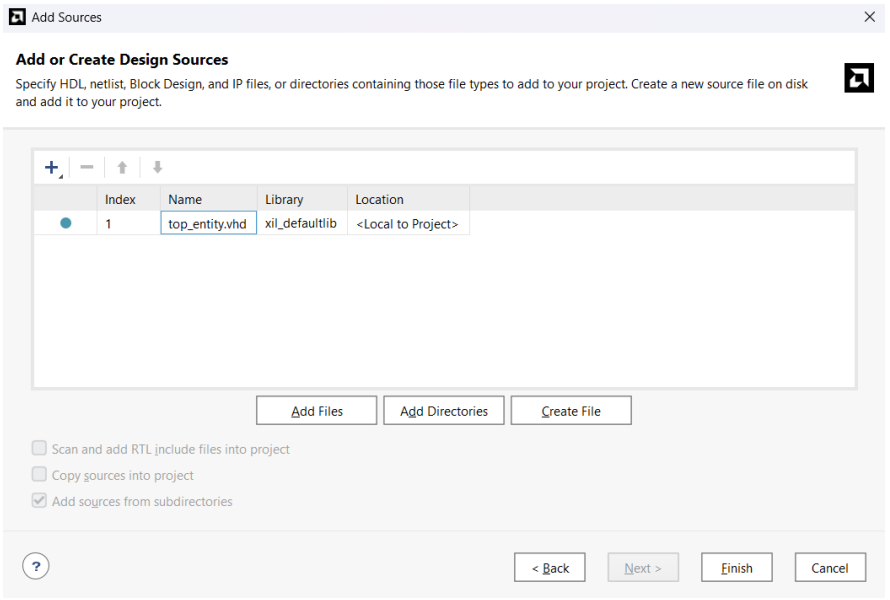


Figura 2: Creación y adición de archivo VHDL al proyecto.

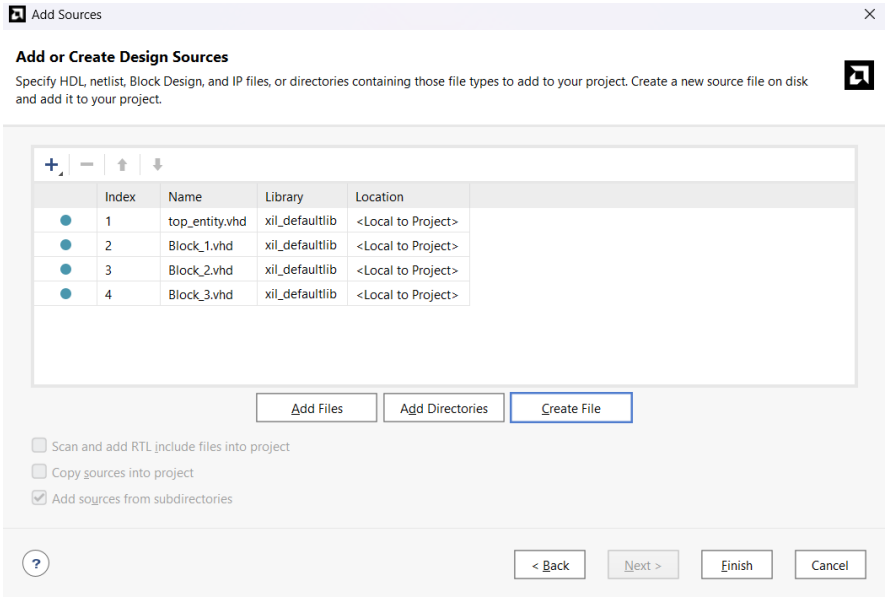


Figura 3: Creación y adición de archivos VHDL al proyecto.

Una vez definida la cantidad de archivos a crear como se menciono se pueden colocar las variables en este panel o mas adelante, ambos métodos son correctos, sin embargo, si no se coloca un código funcional en los archivos no

se pueden definir como entidad principal o top por lo que se sugiere terminar con la codificación y luego designar el archivo main o top

1.3. Designar el archivo principal (*Set as Top*)

1. En el panel `Sources`, localiza `signed_adder.vhd` (o tu top).
2. Clic derecho → `Set as Top`. Comprueba que el icono cambie y que `Hierarchy` muestre a los auxiliares debajo del *top*.

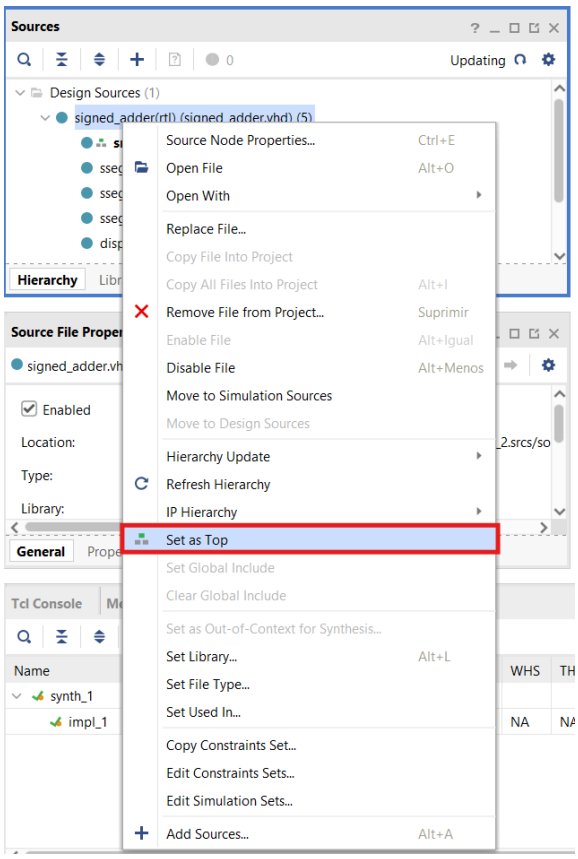


Figura 4: Marcando el archivo principal como *Top*.

2. Descripción del Circuito en VHDL

El diseño se ha dividido en tres módulos principales: **sumador con signo**, **decodificador** y **driver de 7 segmentos**. En esta sección se describe en detalle el primer bloque, encargado de realizar la operación de suma o resta de números con signo.

2.1. Módulo 1: Sumador con signo

El archivo `sum_sign.vhd` implementa un sumador capaz de manejar números con signo, representados en formato de magnitud y signo. El código se presenta a continuación:

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity sum_sign is
6    generic(N : integer := 4);
7    port (
8      a, b : in  std_logic_vector (N-1 downto 0);
9      sum  : out std_logic_vector (N downto 0)
10   );
11 end sum_sign;
12
13 architecture rtl of sum_sign is
14   -- Senales auxiliares
15   signal mag_a, mag_b             : unsigned (N-1 downto
         0);
16   signal max, min, mag_sum        : unsigned (N-1 downto
         0);
17   signal sign_a, sign_b, sign_sum : std_logic;
18 begin
19   mag_a  <= unsigned ('0' & a(N-2 downto 0));
20   mag_b  <= unsigned ('0' & b(N-2 downto 0));
21   sign_a <= a(N-1);
22   sign_b <= b(N-1);
23
24   process (mag_a, mag_b, sign_a, sign_b)
25   begin
26     if mag_a > mag_b then
27       max      <= mag_a;
28       min      <= mag_b;
29       sign_sum <= sign_a;
30     else
31       max      <= mag_b;
32       min      <= mag_a;
33       sign_sum <= sign_b;
34     end if ;
35   end process;
36   mag_sum <= max + min when (sign_a = sign_b) else max -
```

```
      min;
37  sum      <= std_logic_vector (sign_sum & mag_sum);
38 end architecture;
```

Listing 1: Codigo completo sumador

Librerías y entidad

- `ieee.std_logic_1164` define el tipo `std_logic` y sus operaciones lógicas.
- `ieee.numeric_std` permite trabajar con operaciones aritméticas en tipos `unsigned` y `signed`.
- El **genérico** `N` indica el número de bits de los operandos, facilitando escalabilidad.
- Las entradas `a` y `b` son vectores de `N` bits, donde el bit más significativo (`MSB`) representa el **signo**:
  - `0` → número positivo
  - `1` → número negativo
- La salida `sum` tiene `N+1` bits: el primer bit para el signo y el resto para la magnitud.

```
1 architecture rtl of sum_sign is
2   signal mag_a, mag_b             : unsigned (N-1 downto
      0);
3   signal max, min, mag_sum        : unsigned (N-1 downto
      0);
4   signal sign_a, sign_b, sign_sum : std_logic;
5 begin
```

Listing 2: Definicion de Senales

Señales internas

- `mag_a`, `mag_b` almacenan la **magnitud** de los operandos, descartando el bit de signo.
- `max` y `min` se usan para identificar el operando de mayor y menor magnitud, simplificando la lógica de suma/resta.

- `sign_a`, `sign_b` guardan el signo de cada operando.
- `sign_sum` almacena el signo del resultado.

```
mag_a  <= unsigned ('0' & a(N-2 downto 0));
mag_b  <= unsigned ('0' & b(N-2 downto 0));
sign_a <= a(N-1);
sign_b <= b(N-1);
```

Listing 3: Extraccion de magnitud y signo

Extracción de magnitud y signo Se concatenan ceros al inicio (`'0' & ...`) para asegurar que las magnitudes se manejen como valores `unsigned` sin afectar la operación aritmética. El bit más significativo de cada operando se asigna como signo.

```
process (mag_a, mag_b, sign_a, sign_b)
begin
  if mag_a > mag_b then
    max      <= mag_a;
    min      <= mag_b;
    sign_sum <= sign_a;
  else
    max      <= mag_b;
    min      <= mag_a;
    sign_sum <= sign_b;
  end if;
end process;
```

Listing 4: Logica comibacional

Selección de operandos El proceso compara las magnitudes para determinar cuál es mayor (`max`) y cuál menor (`min`). El signo del resultado `sign_sum` se hereda del operando de mayor magnitud, lo que es coherente con la resta de valores con signo.

```
mag_sum <= max + min when (sign_a = sign_b) else max -
    min;
sum     <= std_logic_vector (sign_sum & mag_sum);
```

Listing 5: Operacion Aritmetica

Operación aritmética

- Si ambos operandos tienen el mismo signo (`sign_a = sign_b`), se suman sus magnitudes.
- Si tienen signo diferente, se realiza una resta de magnitudes.
- Finalmente, el resultado se reconstruye como un vector `std_logic_vector` concatenando el bit de signo con la magnitud obtenida.

Este módulo es completamente independiente y podrá ser instanciado en el bloque principal mediante `port map`, permitiendo que otros módulos (como el decodificador y el driver de display) utilicen su salida para mostrar los resultados.

2.2. Módulo 2: Decodificador Hexadecimal a Display de 7 Segmentos

Este módulo, llamado `hex2sseg`, convierte un número hexadecimal de 4 bits en la señal que controla un display de 7 segmentos de ánodo común. Además, recibe una señal `dp` para controlar el punto decimal del display.

El código completo se presenta a continuación:

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity hex2sseg is
  port (
    hex  : in  std_logic_vector (3 downto 0);
    dp   : in  std_logic;
    sseg : out std_logic_vector (7 downto 0)
  );
end hex2sseg;

architecture rtl of hex2sseg is
begin

  with hex select
    sseg(6 downto 0) <=
      "1000000" when "0000", --0
      "1111001" when "0001", --1
      "0100100" when "0010", --2
      "0110000" when "0011", --3
      "0011001" when "0100", --4
      "0010010" when "0101", --5
      "0000010" when "0110", --6
      "1111000" when "0111", --7
      "0000000" when "1000", --8
      "0010000" when "1001", --9
      "0001000" when "1010", --a
      "0000011" when "1011", --b
      "1000110" when "1100", --c
      "0100001" when "1101", --d
      "0000110" when "1110", --e
      "0001110" when others; --f
  sseg(7) <= dp;

end architecture;
```

Listing 6: Decodificador hexadecimal a 7 segmentos

Explicación detallada

1. Librerías y entidad Se importan las librerías estándar para lógica (`std_logic_1164`) y aritmética (`numeric_std`), al igual que en otros módulos.

La entidad `hex2sseg` tiene tres puertos:

- `hex`: entrada de 4 bits que representa un número hexadecimal (de 0 a 15).
- `dp`: entrada para el control del punto decimal (decimal point) del display.
- `sseg`: salida de 8 bits para controlar los segmentos y el punto decimal del display.

2. Arquitectura y sentencia `with ... select` La arquitectura principal está dentro del bloque `begin ... end`. Se usa la sentencia:

```
with hex select sseg(6 downto 0) <= ...;
```

Esta construcción es una forma concisa y clara de asignar valores a un vector dependiendo del valor de una señal de control. Se denomina **selección condicional concurrente**.

3. Sintaxis y funcionamiento del `with ... select` La sintaxis general es:

```
with <senal_control> select
  <senal_destino> <=
    <valor1> when <condicion1>,
    <valor2> when <condicion2>,
    ...,
    <valorN> when others;
```

En este caso, la señal de control es `hex`, un vector de 4 bits. Para cada valor posible de `hex`, se asigna un patrón específico de 7 bits a `sseg(6 downto 0)` que representa qué segmentos se encienden para mostrar el dígito hexadecimal correspondiente en el display.

4. Codificación de los segmentos Cada cadena binaria representa el estado de los segmentos `a` a `g` del display (generalmente en el orden `abcdefg`). Por ejemplo:

- Para `hex = "0000"` (valor decimal 0), se asigna `"1000000"`: El segmento `a` está apagado (1), los demás encendidos (0) — esto debido a que el display es de ánodo común y se activa con 0.
- Para `hex = "0001"` (valor decimal 1), se asigna `"1111001"`, y así sucesivamente.

Estos patrones están diseñados para que al activarse los segmentos correctos se muestre el número o letra hexadecimal correspondiente.

5. La asignación del punto decimal Luego de la selección, se asigna directamente:

```
sseg(7) <= dp;
```

Esto asigna el bit más significativo de `sseg` al control del punto decimal, permitiendo activarlo o desactivarlo según el valor de `dp`.

6. Nota sobre ánodo común Es importante aclarar que en la tarjeta Nexys y en Vivado, el display es de **ánodo común**. Esto implica que los segmentos se activan poniendo el bit en `0` (nivel bajo), y se apagan con `1`. Por ello, los patrones en el código usan esta lógica invertida, donde un 0 en el bit enciende el segmento correspondiente.

2.3. Módulo 3: Driver Multiplexor para Displays de 7 Segmentos

Este módulo, denominado `disp_mux`, implementa un multiplexor para controlar hasta 8 displays de 7 segmentos utilizando un único conjunto de líneas de segmentos. El multiplexado permite reducir la cantidad de pines usados en la tarjeta Nexys y manejar varios dígitos con un solo decodificador.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity disp_mux is
  generic(N : integer := 16);
  port (
    clk   : in std_logic;
    reset : in std_logic;
    in0   : in std_logic_vector (7 downto 0);
    in1   : in std_logic_vector (7 downto 0);
    in2   : in std_logic_vector (7 downto 0);
    in3   : in std_logic_vector (7 downto 0);
    in4   : in std_logic_vector (7 downto 0);
    in5   : in std_logic_vector (7 downto 0);
    in6   : in std_logic_vector (7 downto 0);
    in7   : in std_logic_vector (7 downto 0);
    an    : out std_logic_vector (7 downto 0);
    sseg  : out std_logic_vector (7 downto 0)
  );
end disp_mux;

architecture rtl of disp_mux is
  signal q_reg : unsigned (N-1 downto 0);
  signal q_next : unsigned (N-1 downto 0);
  signal sel : std_logic_vector (2 downto 0);
begin

  process (clk, reset)
  begin
    if reset = '1' then
      q_reg <= (others => '0');
    elsif rising_edge(clk) then
      q_reg <= q_next;
    end if;
  end process;


  q_next <= q_reg + 1;
  sel <= std_logic_vector (q_reg(N-1 downto N-3));


```

```
43   process (sel, in0, in1, in2, in3, in4, in5, in6, in7)
44   begin
45     case sel is
46       when "000" =>
47         an(7 downto 0) <= "11111110";
48         sseg           <= in0;
49       when "001" =>
50         an(7 downto 0) <= "11111101";
51         sseg           <= in1;
52       when "010" =>
53         an(7 downto 0) <= "11111011";
54         sseg           <= in2;
55       when "011" =>
56         an(7 downto 0) <= "11110111";
57         sseg           <= in3;
58       when "100" =>
59         an(7 downto 0) <= "11101111";
60         sseg           <= in4;
61       when "101" =>
62         an(7 downto 0) <= "11011111";
63         sseg           <= in5;
64       when "110" =>
65         an(7 downto 0) <= "10111111";
66         sseg           <= in6;
67       when "111" =>
68         an(7 downto 0) <= "01111111";
69         sseg           <= in7;
70       when others =>
71         an(7 downto 0) <= "11111111";
72         sseg           <= (others => '1');
73     end case;
74   end process;
75
76 end architecture;
```

Listing 7: Driver multiplexor para displays de 7 segmentos

Explicación

1. Propósito del módulo El módulo `disp_mux` se encarga de activar de forma secuencial cada uno de los 8 displays conectados a la tarjeta Nexys, enviando la información correcta a mostrar. Esto se logra gracias a la multiplexación, que alterna rápidamente la activación de cada display, dando la

apariencia de que todos están encendidos simultáneamente.

2. Entradas y salidas

- `clk`: señal de reloj que marca el ritmo del multiplexado.
- `reset`: señal para reiniciar el contador interno.
- `in0` a `in7`: vectores de 8 bits que contienen la información para cada uno de los displays (los segmentos y el punto decimal).
- `an`: vector de 8 bits que controla la activación de cada ánodo común de los displays.
- `sseg`: vector de 8 bits que controla los segmentos comunes a todos los displays.

3. Contador interno Se declara una señal `q_reg` de tipo `unsigned` con un ancho definido por el genérico `N` (por defecto 16 bits). Esta señal funciona como un contador que se incrementa en cada ciclo de reloj, y es reseteada con la señal `reset`.

```
process (clk, reset)
begin
  if reset = '1' then
    q_reg <= (others => '0');
  elsif rising_edge(clk) then
    q_reg <= q_next;
  end if;
end process;

q_next <= q_reg + 1;
```

El contador va desde 0 hasta $2^N - 1$, incrementándose en cada pulso de reloj.

4. Selección del display activo Los tres bits más significativos del contador se utilizan para seleccionar cuál de los ocho displays activar. Esto se hace asignando a la señal `sel` los bits `q_reg`($N-1$ downto $N-3$):

```
sel <= std_logic_vector (q_reg(N-1 downto N-3));
```

Así, `sel` toma valores de `"000"` a `"111"` en secuencia, lo que permite multiplexar los 8 displays.

5. Proceso de multiplexación Un proceso sensible a `sel` y las entradas `in0` a `in7` selecciona, mediante una sentencia `case`, qué display activar y qué datos enviar a los segmentos:

- Se activa un solo ánodo colocando en `an` un vector con todos bits en '1' excepto el bit correspondiente al display activo, que se pone en '0' (ánodo común activo en bajo).
- Se asigna a `sseg` el valor de la entrada correspondiente para ese display.
- El caso `others` garantiza que si `sel` tiene un valor fuera de rango, todos los displays se apagan.

```
process (sel, in0, in1, in2, in3, in4, in5, in6, in7)
begin
  case sel is
    when "000" =>
      an(7 downto 0) <= "11111110";
      sseg           <= in0;
    when "001" =>
      an(7 downto 0) <= "11111101";
      sseg           <= in1;
    when "010" =>
      an(7 downto 0) <= "11111011";
      sseg           <= in2;
    when "011" =>
      an(7 downto 0) <= "11110111";
      sseg           <= in3;
    when "100" =>
      an(7 downto 0) <= "11101111";
      sseg           <= in4;
    when "101" =>
      an(7 downto 0) <= "11011111";
      sseg           <= in5;
    when "110" =>
      an(7 downto 0) <= "10111111";
      sseg           <= in6;
    when "111" =>
      an(7 downto 0) <= "01111111";
      sseg           <= in7;
    when others =>
      an(7 downto 0) <= "11111111";
      sseg           <= (others => '1');
  end case;
end process;
```

6. Consideraciones finales

- El multiplexado debe ser suficientemente rápido para que el ojo humano perciba todos los displays encendidos simultáneamente.
- El módulo se puede escalar cambiando el valor del genérico `N`, pero se debe ajustar la lógica para que la selección `sel` corresponda correctamente a la cantidad de displays.

3. Archivo principal: Integración de módulos y uso de `port map`

El archivo principal `signed_adder.vhd` es el encargado de integrar los tres módulos desarrollados previamente: el sumador con signo (`sum_sign`), el decodificador hexadecimal a 7 segmentos (`hex2sseg`) y el multiplexor de displays (`disp_mux`). Este diseño utiliza la estructura jerárquica de VHDL, donde cada módulo es una entidad independiente que se instancia y conecta mediante `port map`.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity signed_adder is
6    generic(N : integer := 16);
7    port (
8      clk  : in  std_logic;
9      sw   : in  std_logic_vector (7 downto 0);
10     an   : out std_logic_vector (7 downto 0);
11     sseg : out std_logic_vector (7 downto 0)
12   );
13 end signed_adder;
14
15 architecture rtl of signed_adder is
16   signal sum_m,aout,a_m,bout,b_m: std_logic_vector (3
       downto 0);
17   signal sum: std_logic_vector (4 downto 0);
18
19   signal in_i3,in_i2,in_i1,in_i0  : std_logic_vector (7
       downto 0);
20   signal in_d3,in_d2,in_d1,in_d0  : std_logic_vector (7
       downto 0);
```

```
21
22 begin
23   aout <= sw(3 downto 0);
24   bout <= sw(7 downto 4);
25
26   sm_adder_unit : entity work.sum_sign (rtl)
27     generic map(N => 4)
28     port map(
29       a   => aout,
30       b   => bout,
31       sum => sum
32     );
33
34   sum_m <= sum(3 downto 0);
35   sseg_unit_1 : entity work.hex2sseg (rtl)
36     port map(hex => sum_m, dp => '1', sseg => in_d1);
37
38   in_d2 <= "10111111" when sum(4) = '1' else
39            "11111111";
40
41
42   a_m <= '0' & aout(2 downto 0);
43   b_m <= '0' & bout(2 downto 0);
44   sseg_unit_2 : entity work.hex2sseg (rtl)
45     port map(hex => a_m, dp => '1', sseg => in_i2);
46   sseg_unit_3 : entity work.hex2sseg (rtl)
47     port map(hex => b_m, dp => '1', sseg => in_i0);
48
49   in_i3 <= "10111111" when aout(3) = '1' else
50            "11111111";
51   in_i1 <= "10111111" when bout(3) = '1' else
52            "11111111";
53
54   in_d3 <= "11111111";
55   in_d0 <= "11111111";
56
57   disp_unit : entity work.disp_mux (rtl)
58     generic map(N => N)
59     port map(
60       clk   => clk,
61       reset => '0',
62       in0   => in_d0,
63       in1   => in_d1,
```

```
64        in2    => in_d2,
65        in3    => in_d3,
66        in4    => in_i0,
67        in5    => in_i1,
68        in6    => in_i2,
69        in7    => in_i3,
70        an     => an,
71        sseg   => sseg
72      );
73
74  end architecture;
```

Listing 8: Archivo principal que integra módulos con `port map`

Análisis y explicación

1. Jerarquía y modularidad en VHDL VHDL permite construir diseños jerárquicos donde módulos independientes (entidades) se integran en diseños más complejos. Cada módulo tiene su propia entidad y arquitectura, y para reutilizarlos en otro diseño se instancian dentro de la arquitectura superior.

Esto promueve la reutilización, claridad, y facilita el mantenimiento del código.

2. La instancia mediante `port map` La palabra clave `port map` se utiliza para conectar formalmente los puertos de la entidad instanciada con señales o puertos del módulo superior.

Por ejemplo, esta instancia:

```
1  sm_adder_unit : entity work.sum_sign (rtl)
2    generic map(N => 4)
3    port map(
4      a   => aout,
5      b   => bout,
6      sum => sum
7    );
```

Hace lo siguiente:

- Declara una instancia llamada `sm_adder_unit` del módulo `sum_sign`, arquitectura `rtl` del proyecto `work` (proyecto local).
- Usa `generic map` para asignar el parámetro genérico `N = 4`, configurando el tamaño del sumador.

- Con `port map` conecta el puerto `a` del sumador con la señal local `aout`, `b` con `bout` y la salida `sum` con la señal `sum` declarada localmente.

3. Importancia del `port map` El `port map` es crucial porque:

- Define explícitamente qué señales del nivel superior se conectan a los puertos del módulo instanciado.
- Permite reutilizar un mismo módulo con diferentes conexiones y parámetros, haciendo el diseño flexible.
- Clarifica el flujo de datos y facilita la lectura del diseño jerárquico.

4. Otros usos de `port map` en el archivo El archivo también instancia tres veces el módulo `hex2sseg` para mostrar el resultado y los operandos, asignando las señales adecuadas en cada llamada:

```
sseg_unit_1 : entity work.hex2sseg (rtl)
  port map(hex => sum_m, dp => '1', sseg => in_d1);

sseg_unit_2 : entity work.hex2sseg (rtl)
  port map(hex => a_m, dp => '1', sseg => in_i2);

sseg_unit_3 : entity work.hex2sseg (rtl)
  port map(hex => b_m, dp => '1', sseg => in_i0);
```

Finalmente, el multiplexor `disp_mux` también se instancia con su genérico y puertos conectados, integrando la visualización final:

```
disp_unit : entity work.disp_mux (rtl)
  generic map(N => N)
  port map(
    clk   => clk,
    reset => '0',
    in0   => in_d0,
    in1   => in_d1,
    in2   => in_d2,
    in3   => in_d3,
    in4   => in_i0,
    in5   => in_i1,
    in6   => in_i2,
    in7   => in_i3,
    an    => an,
    sseg  => sseg
  );
```

5. Conexión de señales internas Se utilizan señales internas para adaptar y conectar adecuadamente las salidas y entradas de los módulos:

- `aout` y `bout` extraen partes del vector `sw` para los operandos.
- `sum` guarda el resultado extendido (5 bits) del sumador con signo.
- Se crean señales intermedias para la conversión y el control del punto decimal, por ejemplo `in_d2` y `in_i3` que activan el segmento del signo.

Esta estructura modular y el uso del `port map` permite diseñar sistemas complejos manteniendo claridad y flexibilidad.

4. Diagrama de bloques y modularidad del diseño

La implementación del sumador con signo y su visualización en displays de 7 segmentos se ha dividido en tres bloques funcionales principales, que se integran jerárquicamente en el archivo principal. Esta división modular permite reducir la complejidad y el tamaño del código final, facilitando la comprensión y mantenimiento del diseño.

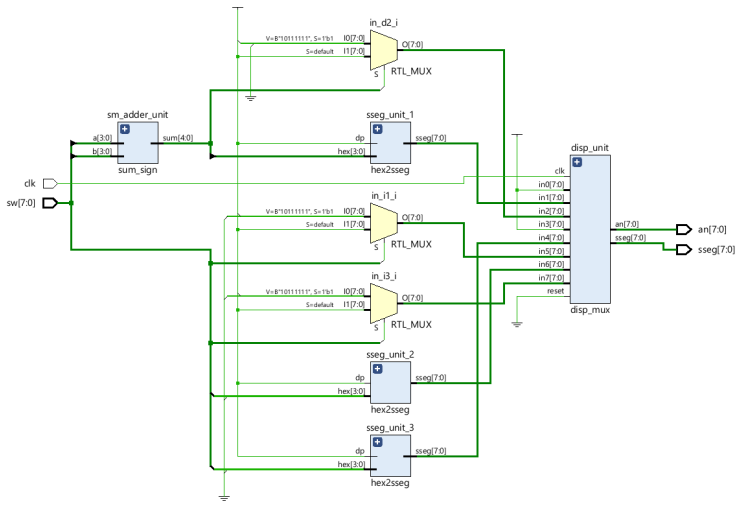


Figura 5: Diagrama de bloques general del sistema `signed_adder`

El diagrama de bloques de la figura 5 muestra cómo el bloque principal instancia tres módulos: el sumador con signo (`sum_sign`), el decodificador

hexadecimal a 7 segmentos (`hex2sseg`) y el multiplexor para los displays (`disp_mux`). Esta arquitectura jerárquica permite un diseño limpio y escalable.

4.1. Bloques funcionales individuales

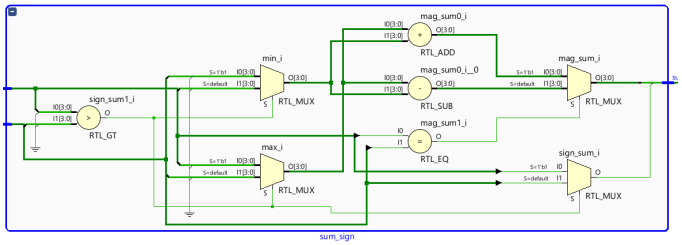


Figura 6: Bloque `sum_sign`: Sumador con signo y manejo de magnitud y signo

El bloque `sum_sign` realiza la suma o resta de dos operandos con signo, separando la magnitud y el bit de signo, y generando el resultado extendido. Este bloque encapsula la lógica aritmética central.

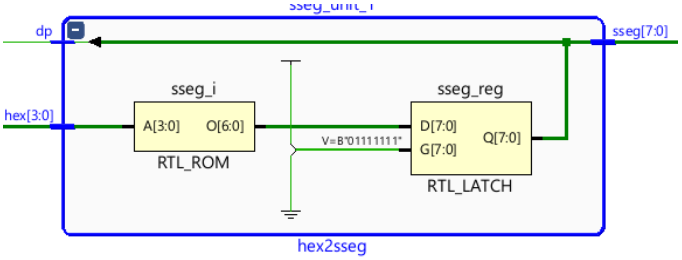


Figura 7: Bloque `hex2sseg`: Decodificador hexadecimal a 7 segmentos

El bloque `hex2sseg` transforma un valor hexadecimal de 4 bits en el patrón de activación para un display de 7 segmentos de ánodo común, incluyendo el control del punto decimal.

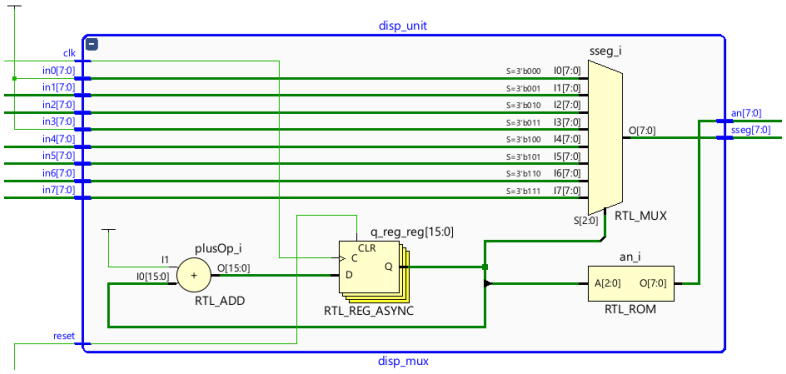


Figura 8: Bloque `disp_mux`: Multiplexor para controlar múltiples displays

Finalmente, el bloque `disp_mux` controla la activación secuencial de los displays mediante multiplexación, reduciendo la cantidad de pines necesarios y permitiendo mostrar múltiples dígitos con un solo decodificador.

4.2. Ventajas de la modularidad

- **Reducción del tamaño del archivo principal**: Al encapsular funciones específicas en módulos independientes, el archivo `signed_adder.vhd` se simplifica, enfocándose solo en la conexión entre bloques y no en la implementación detallada.
- **Reutilización de código**: Cada bloque puede reutilizarse en otros proyectos o diseños sin modificar el archivo principal.
- **Facilidad de mantenimiento y depuración**: Al aislar la funcionalidad en bloques, se puede probar y corregir cada módulo por separado.
- **Escalabilidad**: Permite ampliar el sistema agregando más módulos o modificando existentes sin alterar la estructura global.

5. Asignación de pines en la placa Nexys A7 100T

Para implementar correctamente el diseño en la placa Nexys A7 100T, es fundamental asignar los pines físicos correspondientes a las señales del circuito. A continuación se presenta la configuración recomendada para los pines utilizados en este proyecto, considerando la conexión de los switches, el reloj y los displays de 7 segmentos.

5.1. Señales de entrada

- **Botones y switches:** Se utilizan los switches `SW7` a `SW0` para ingresar los operandos al sumador. En la Nexys A7 100T, estos switches están conectados a los siguientes pines FPGA (ver tabla 1).
- **Reloj (`clk`):** Se utiliza la señal de reloj del oscilador principal de la placa, típicamente a 100 MHz, conectada al pin `E3` (señal `clk_100MHz` en la Nexys A7 100T).

Switch	Pin FPGA	Descripción
`SW0`	J15	Switch 0
`SW1`	L16	Switch 1
`SW2`	M13	Switch 2
`SW3`	R15	Switch 3
`SW4`	R17	Switch 4
`SW5`	T18	Switch 5
`SW6`	U18	Switch 6
`SW7`	R13	Switch 7

Cuadro 1: Asignación de pines para los switches en Nexys A7 100T

5.2. Señales de salida

- **Displays de 7 segmentos:** La salida se conecta a los displays de 7 segmentos integrados en la placa, que constan de 8 dígitos con ánodo común. La señal `an` controla cuál display está activo, y `sseg` controla los segmentos y el punto decimal.

  La asignación de pines para los ánodos y segmentos se muestra en la tabla 2.

Señal	Pin FPGA	Descripción
an[0]	J17	Ánodo display 0
an[1]	J18	Ánodo display 1
an[2]	T9	Ánodo display 2
an[3]	J14	Ánodo display 3
an[4]	P14	Ánodo display 4
an[5]	T14	Ánodo display 5
an[6]	K2	Ánodo display 6
an[7]	U13	Ánodo display 7
sseg[0]	T10	Segmento A
sseg[1]	R10	Segmento B
sseg[2]	K16	Segmento C
sseg[3]	K13	Segmento D
sseg[4]	P15	Segmento E
sseg[5]	T11	Segmento F
sseg[6]	L18	Segmento G
sseg[7]	H15	Punto decimal

Cuadro 2: Asignación de pines para los displays de 7 segmentos en Nexys A7 100T

5.3. Consideraciones adicionales

- En el archivo `XDC` (constraints) se deben declarar estas asignaciones para que Vivado las reconozca al sintetizar y generar el bitstream.
- El diseño asume que los displays son de ánodo común, por lo que los ánodos se activan con un nivel bajo (0).
- El reloj utilizado debe coincidir con la frecuencia del oscilador en la placa para un funcionamiento correcto del multiplexor.

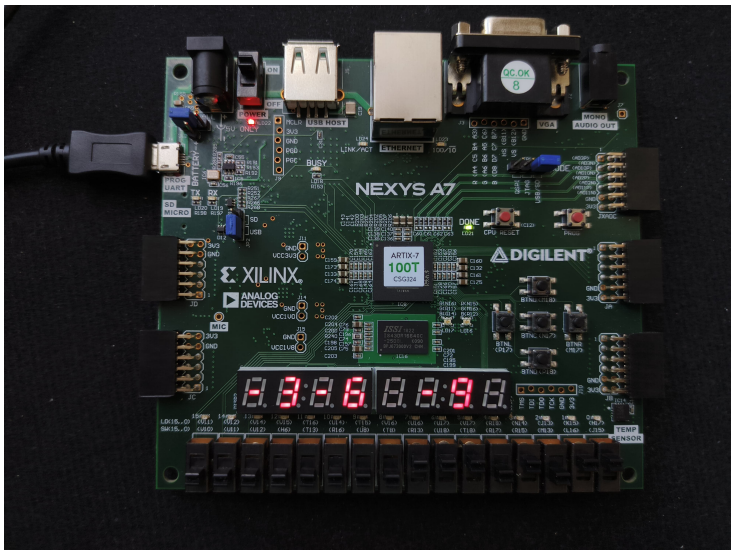
6. Resultados en la placa Nexys A7-100T

Para la validación del diseño implementado en la FPGA Nexys A7-100T, se cargó el bitstream generado en la etapa de implementación. Posteriormente, se realizaron pruebas con diferentes combinaciones de entradas para verificar el correcto funcionamiento del sumador/restador con números con signo.

A continuación, se presentan tres casos de prueba representativos junto con las imágenes capturadas de la placa en funcionamiento:

Caso 1: Suma de dos números negativos

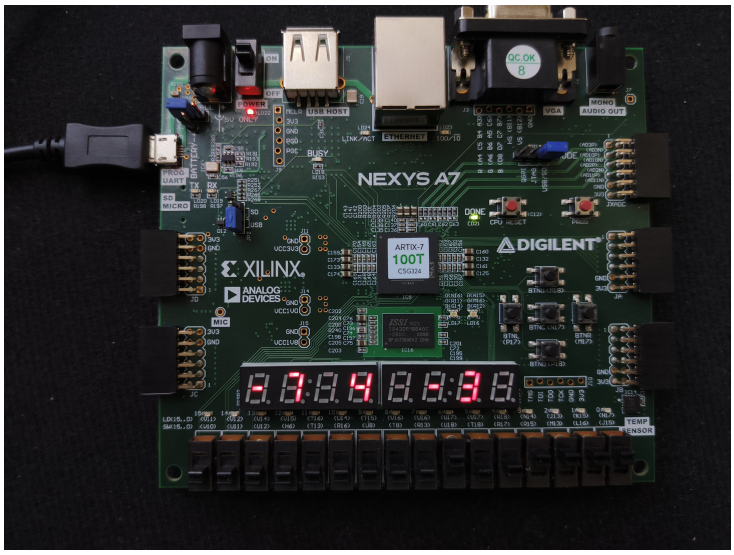
En este escenario, se ingresaron dos números negativos utilizando los interruptores (SW) como entradas binarias en complemento a dos. El resultado obtenido se visualizó correctamente en los displays de siete segmentos, confirmando la correcta interpretación de los valores negativos y su suma.



Resultado esperado: La suma de $-3 + (-6)$ da -9, valor que se muestra en el display.

Caso 2: Suma de un número positivo y uno negativo

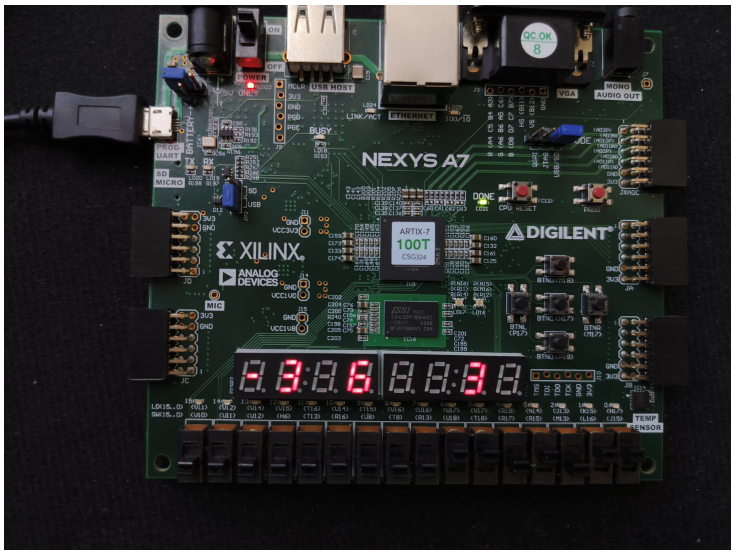
Se configuró el primer operando como negativo y el segundo como positivo. El resultado fue visualizado correctamente, mostrando que el circuito realiza la operación aritmética teniendo en cuenta la magnitud y signo de ambos números.



Resultado esperado: La suma de $-7 + 4$ es -3, valor que se muestra en el display.

Caso 3: Suma donde el negativo es mayor en magnitud

En este caso, el número negativo tiene menor valor absoluto que el positivo, resultando en un valor final positivo. La visualización y el cálculo fueron correctos.



Resultado esperado: La suma de $-3 + 6$ es 3, valor que se muestra en el display.